BitLedger value encoding, scaling factors, and SF selection

Technical reference and design rationale

Document type: internal engineering monograph
Audience: implementers, auditors, and researchers extending BitLedger's monetary value path
Scope: Layer 2–3 value semantics as implemented in this repository (`encode_value`, `decode_value`, SF search in the CLI), not full protocol stack or hardware bit layouts except where they constrain mathematics

---

1. Introduction

BitLedger stores monetary amounts on a fixed-width record by mapping a **true economic value** (represented exactly as a Python `Decimal` at the boundary) into a **small integer** N together with **metadata** that describes scale, split, rounding, and optional quantity semantics. The mapping is not injective: many triples (SF, D, N) can represent the same nominal amount; conversely, a single true value may admit **no** exact representation for some choices of parameters, forcing **controlled rounding** that must be disclosed on the wire.

This document does three things:

1. **Teaches** the algebra of the value path in a form suitable for proofs and test design.
2. **Records** engineering trials and the choices that led to the current behaviour (128-entry scaling table, exact-first SF search, legacy compatibility flag).
3. **States implications** for how the tool and any downstream system should treat encodings, errors, and user expectations.

The implementation lives primarily in:

- `bitledger/encoder.py` — `SCALING_FACTORS`, `encode_value`, `decompose`, rounding sets
- `bitledger/decoder.py` — `decode_value`
- `bitledger/cli_make.py` — `find_smallest_sf`, `resolve_encoding_plan`
- `bitledger/cli.py` — `encode --auto-sf` and defaults

---

2. Wire model and notation

2.1 Scalar amount on decode (non-quantity path)

Let:

- $v \in \mathbb{Q}_{\text{Decimal}}$ be the **true** monetary amount in natural units (e.g. dollars), supplied as `Decimal`.

- $\mathsf{SF} = 10^k$ with **integer** $k \in \{0, \ldots, K-1\}$ the scaling factor associated with wire index `sf_index`. In this codebase $K = 128$ and $\mathsf{SF}_k = 10^k$.

- $D \in \{0, 1, \ldots, 6\}$ be the **decimal position** interpreted as "divide by 10^D on decode" (wire code `decimal_position`; the reserved code `111` is rejected).

- $S \in \{0, \ldots, 17\}$ be the **optimal split** for decomposing N into high and low parts.

- A and r be the **multiplicand** and **multiplier** limbs such that, when `quantity_present` is false,

$$N = A \cdot 2^S + r, \quad 0 \le r < 2^S.$$

The decoded wire value is

$$\hat{v} = \frac{N \cdot \mathsf{SF}_k}{10^D} = N \cdot 10^{k-D}.$$

This matches `decode_value` in `decoder.py`: `num = Decimal(N) * SF` and `den = Decimal(10) ** D`.

2.2 Encode map (idealised)

Encoding chooses k and integral N such that the **rounded** discrete value matches protocol rules. Define the **scaled rational** before rounding:

$$R_k(v) = \frac{v \cdot 10^D}{\mathsf{SF}_k} = v \cdot 10^{D-k}.$$

If $R_k(v) \in \mathbb{Z}$ (as `Decimal` equality to its integral value in the implementation), then

$$N = R_k(v), \quad \mathsf{rf} = 0, \quad \mathsf{rd} = 0.$$

Otherwise N is obtained by a **mode-dependent** rounding of $R_k(v)$ to an integer, with $\mathsf{rf} = 1$ and rd recording direction semantics for bits 26–27.

2.3 Hard capacity constraint

Regardless of k, the wire integer must satisfy

$$0 \le N \le N_{\max}, \quad N_{\max} = 33\,554\,431 = 2^{25} - 1.$$

If rounding yields $N > N_{\max}$, `encode_value` raises `EncoderError`. This is the **dominant long-run bottleneck** for representable magnitude: it is **not** lifted by increasing k alone, because although larger k shrinks $R_k(v)$ for fixed v, the **product** $N \cdot 10^{k-D}$ must still reconstruct the decoded amount; the ladder trades **resolution** against **overflow** in R_k space, not against the **bit width** of N.

---

3. Rounding as a signed functional

The encoder implements three regimes keyed by **account pair** nibbles:

- **Round up** on certain pairs (`ROUND_UP_PAIRS`).
- **Round down** on others (`ROUND_DOWN_PAIRS`).
- **Round half up** (banker-style tie-breaking toward $+\infty$ for half-integers) on the remainder (`nearest`).

Formally, let $\lfloor x \rfloor$, $\lceil x \rceil$, and $\mathrm{nint}(x)$ denote floor, ceiling, and half-up to integer. The implementation sets:

- **Up:** $N = \lceil R \rceil$, $\mathsf{rd} = 1$, $\mathsf{rf} = 1$ when $R \notin \mathbb{Z}$.
- **Down:** $N = \lfloor R \rfloor$, $\mathsf{rd} = 0$, $\mathsf{rf} = 1$.
- **Nearest half-up:** $N = \mathrm{nint}(R)$; $\mathsf{rd} = 1$ iff $N > R$, else 0, with $\mathsf{rf} = 1$ when $R \notin \mathbb{Z}$.

Invariant (decode vs encode): when $\mathsf{rf} = 1$, the decoded $\hat{v}$ generally **differs** from v; the CLI labels this as "typed amount $\geq$ decoded" or "$\leq$ decoded" depending on rd, aligning user mental models with one-sided error.

Design choice — `Decimal` only at the boundary: floating-point is rejected at `encode_value` entry. This avoids binary base-2 artefacts in a **decimal monetary** ladder and makes $R_k(v)$ integrality tests meaningful.

---

4. The scaling-factor ladder

4.1 Geometric progression

For each index k, $\mathsf{SF}_k = 10^k$. The map $k \mapsto \mathsf{SF}_k$ is a **uniform geometric sequence** in the logarithmic domain:

$$\log_{10} \mathsf{SF}_k = k.$$

Thus each increment in k **divides** $R_k(v)$ by 10 (for fixed v, D), which is why large nominal magnitudes eventually "fit" into $N \leq N_{\max}$ if k is large enough—**provided** the rounded integer still lies in range.

4.2 Alignment with Layer 2

Layer 2 carries a **7-bit** scaling factor index (`0..127`). Prior to extending `SCALING_FACTORS`, the table exposed only **18** entries (`0..17`). Indices `18..127` were valid on the wire in principle but **not** usable through `encode_value` / `decode_value` without ad hoc extension or error.

Engineering choice: extend the table to **128** entries so that **every wire-legal index** has a defined SF_k in Python. This:

- removes an artificial **breadth** gap between "what the wire can name" and "what the reference encoder accepts";

- keeps **one source of truth** (`len(SCALING_FACTORS)`) for bounds checks in encoder and decoder.

Non-choice (explicitly out of scope here): widening N beyond 25 significant bits would require a **record format change**—bits stolen from elsewhere in the 40-bit Layer 3 record or beyond. The trials discussed here **preserve** the monetary capacity envelope $N_{\max}$ and instead exploit the **SF dimension** and **search policy**.

---

5. SF selection as an optimisation problem

When the user does not fix `--sf`, the CLI must choose $k \in [k_{\min}, k_{\max}]$ such that `encode_value` succeeds. Two objectives have been used in practice:

5.1 Legacy: first feasible index (ascending scan)

Policy L (legacy): return the smallest k in ascending order for which `encode_value` does not raise.

Merit: matches an intuitive "use the coarsest scale that still fits" heuristic when "fits" was implicitly tied to the first successful R_k that cleared overflow.

Demerit: the first feasible k may force $\mathsf{rf} = 1$ even when a **larger** k would yield $\mathsf{rf} = 0$ (exact). That is suboptimal for **precision** and for downstream workflows that reject rounding unless `--accept-rounding`.

Empirical note: for many natural amounts and parameter ranges tested during development, **L** and **exact-first** (below) often agree, because overflow failures at small k frequently defer the first success until R_k is already integral. **Agreement is not guaranteed** by mathematics; it is a property of the interaction between v, D, pair rounding, and $N_{\max}$.

5.2 Default today: exact-first, then smallest feasible

Policy E (exact-first):

1. Among $k \in [k_{\min}, k_{\max}]$ ascending, choose the **smallest** k such that encoding succeeds **and** $\mathsf{rf} = 0$.

2. If none, fall back to **Policy L** on the same interval.

Implementation note: the code performs this as **two ascending passes** over the same bounded range (at most 128 iterations per pass), which is $O(k_{\max} - k_{\min})$ with trivial constant factors—acceptable for a CLI planner.

Merit: maximises the chance that `make` / `check-amount` / `encode --auto-sf` propose a plan that is **exact on the wire** without user hand-tuning of k, whenever the interval admits such a k.

Demerit: when no exact k exists in the interval, behaviour coincides with **L**; when exact exists only at larger k, the chosen SF is **less "coarse"** than the legacy first-success index would have

been—**intentionally**, because coarseness traded for rounding is often undesirable in ledgering.

Compatibility: `--legacy-sf-search` restores **L** for regression, cross-tool parity, or deliberate "smallest k even if rounded" workflows.

---

6. Trials and falsifiable hypotheses encountered in design

This section records **what was tried** or **what was hypothesised**, and **what followed**.

6.1 Hypothesis: "First success equals smallest exact when exact exists"

False in general. Constructive counterexamples exist in principle whenever:

- for some k_1, $R_{k_1} \notin \mathbb{Z}$ but rounded $N_1 \leq N_{\max}$ (success with $\mathsf{rf} = 1$);
- for some $k_2 > k_1$, $R_{k_2} \in \mathbb{Z}$ and $N_2 \leq N_{\max}$ (success with $\mathsf{rf} = 0$).

Whether such pairs appear inside **practical** $[k_{\min}, k_{\max}]$ for real profiles is an **empirical** question; the **policy** is nonetheless justified by worst-case correctness, not by frequency of divergence.

6.2 Trial: brute-force divergence search (small ranges)

Automated searches over coarse grids of amounts, pairs, and S often found **no** divergence between **L** and **E** within narrow SF caps—suggesting that for many "nice" decimals and default caps, the policies align. That empirical outcome **must not** be mistaken for a theorem.

6.3 Trial: extending the SF table without touching $N_{\max}$

Chosen path. Extending `SCALING_FACTORS` to 128 entries changes **which** (v, k) pairs are well-defined, but not the **cardinality** of representable N. Thus:

- **Breadth** in **nominal space** increases because larger k allows smaller R_k before overflow.
- **Precision** at a fixed v can improve because **E** may select a k that clears integrality of R_k.

6.4 Trial: default `--max-sf` raised from 17 to 127

Rationale: if the wire allows $k \leq 127$, defaulting the planner's upper bound to the same maximum avoids silent failure modes where a user's amount is representable in-protocol but **not discoverable** by auto-search under a low cap.

Operational implication: scripts that assumed "search only to 17" now explore a larger space by default. That is **usually** desirable; users who want the old cap can pass `--max-sf 17`.

---

7. Worked examples (numeric)

7.1 Billion-class amount (exact at moderate k)

Let $v = 2.5 \times 10^9$, $D = 2$, pair and S as in repository tests (`pair=4`, `S=8`). Then

$$R_k(v) = v \cdot 10^{2-k} = 2.5 \times 10^{11-k}.$$

For $k = 3$, $R_3 = 2.5 \times 10^8 > N_{\max} \to$ overflow after rounding to integer.
For $k = 4$, $R_4 = 2.5 \times 10^7 = 25\,000\,000 \leq N_{\max}$, integral $\to$ exact.

Both **L** and **E** return $k = 4$ here: first success is already exact.

7.2 Extreme magnitude with extended ladder

Take $v = 10^{98}$, $D = 2$. Then

$$R_k(v) = 10^{98} \cdot 10^{2-k} = 10^{100-k}.$$

Overflow is equivalent to $10^{100-k} > N_{\max}$. The smallest integer k with $10^{100-k} \leq 33\,554\,431$ is $k = 93$ (since $10^7 \leq N_{\max} < 10^8$). At $k = 93$, R is integral and $N = 10^7$; encoding is exact. This example appears in tests as a **regression anchor** for "table breadth matters."

---

8. Implications for BitLedger operation

8.1 CLI planning vs strict encode

`make` and `check-amount` expose full numeric detail including rf and suggested `encode` argv. **`encode` still refuses** rounded encodings unless `--accept-rounding` is set—even when auto-SF finds a plan—preserving a **consent** step for non-exact wire states.

Exact-first reduces spurious refusals: users are more often presented with a plan where $\mathsf{rf} = 0$, so the **same** flags succeed at `encode` without extra overrides.

8.2 Decoder symmetry

`decode_value` rejects `sf_index` outside `len(SCALING_FACTORS)`. After extension, **more stored records** become decodable by the reference implementation without treating SF as opaque. Cross-version **interoperability** still requires agreement on K: older senders with $k > 17$ on the wire were already non-interoperable with a $K = 18$ table; raising K to 128 **narrows** that gap relative to the wire.

8.3 Testing strategy

The repository's roundtrip test parametrizes **all** SF indices with a constructed exact amount $v_k = \mathsf{SF}_k/10^D$ at fixed $D = 2$, ensuring $\mathsf{rf} = 0$ when encodable. This is a **uniform structural test** over the ladder, not merely spot checks.

---

9. Mathematical invariants and proof obligations

Implementers may wish to formalise the following lemmas (sketches only):

Lemma (decode re-encode for exact wires). If $\mathsf{rf} = 0$ and quantity mode is off, then `decode_value` inverts `encode_value` for the same (k, D, S) and pair **when** no overflow occurs and decimal arithmetic remains exact in `Decimal`.

Lemma (monotonicity of $|R_k|$ in k). For fixed $v > 0, D$, $|R_k(v)|$ is strictly decreasing in k along the geometric ladder. Thus overflow failure at k implies eventual feasibility for large enough k **iff** integrality and rounding do not re-expand N beyond $N_{\max}$—rare pathologies can still exist for adversarially chosen v under half-up/down asymmetries; production systems should still treat `EncoderError` as reachable.

Invariant (bit semantics). $\mathsf{rf} = 0 \Rightarrow \mathsf{rd} = 0$ in the encoder's exact branch; combinations labelled INVALID in CLI strings must never be emitted by this encoder path.

---

10. Pedagogical summary

1. **Value encoding is a constrained inverse problem:** find integers (N, k) (and splits) so that $\hat{v}$ approximates v under explicit rounding laws and hard bounds.

2. **The SF index is a discrete zoom control** on $R_k(v)$; extending the table aligns the zoom range with the wire.

3. **Exact-first SF search** is a **lexicographic optimisation** (rf, k) with a tie-break that prefers $\mathsf{rf} = 0$ then minimal k, except when no exact exists—then minimal k among feasible rounded encodings (same as legacy).

4. **The dominant economic capacity limit remains** $N_{\max}$; SF extension improves **reach** into large-magnitude or high-precision regimes **subject** to that ceiling.

---

11. Suggested citations in code review

When auditing changes to this subsystem, reviewers should verify:

- `encoder.SCALING_FACTORS` length matches Layer 2 bit width policy.
- `encode_value` and `decode_value` use the same upper bound (`len(SCALING_FACTORS)`).
- `find_smallest_sf` two-pass logic preserves **E** vs **L** semantics under `--legacy-sf-search`.
- CLI defaults for `--max-sf` remain consistent across `make`, `check-amount`, `suggest-sf`, and `encode --auto-sf`.

---

12. Closing remark

A "legendary" reference is not only precise but **honest about limits**: BitLedger's value path here is a **finite discrete** system approximating **continuous** economic magnitudes. The scaling ladder, rounding disclosure, and SF search policy are the **moral and mathematical** interface between

those worlds. This document is intended to remain the canonical narrative for that interface as the codebase evolves.